

Founding AI Engineer Assignment — Meka . Durga sai vardhan reddy

Repo: <https://github.com/mdsvr/tokenfold-bench> **Live:** <https://tokenfold-bench.vercel.app>

1. What I built and why

I built **TokenFold Bench** — a tool that measures how much of a codebase you actually need to send a model to get a correct answer.

The assignment said build anything, and the examples given (quiz app, FPV game, collab app) had nothing to do with Superbrain. I thought about that for a while. I could build a polished CRUD app and it would say nothing about whether I understood the product. But section 1 says understand the product deeply, and section 3 asks me to critique it. I didn't want to critique a context engine based on marketing copy. So I built one, in a day, to find out where the hard parts actually are.

To be clear about what this is: **it is not a competing engine**. It is the instrument. It is the thing that tells you whether a context engine is working, which as far as I can tell nobody currently ships — including Superbrain.

What it does

Pick a repo, ask a question, and it compares three ways of feeding code to a model:

strategy	tokens	cost (projected)	what the model can see
naive — send everything	408,333	\$2.04	all 400 files
retrieval — top-k keyword	39,966	\$0.20	17 files, 383 invisible
folded — tiered resolution	39,996	\$0.20	10 full + 3 sigs + 387 names, 0 invisible

Measured on `django/django`, 400 files indexed, asking "how does user authentication and password hashing work". I picked Django because that is the repo Superbrain benchmarks against on their own site.

Then it actually answers the question with each context, so you can check the compressed version is still correct. It is — folded gets PBKDF2 / `pbkdf2_sha256`, 1,800,000 iterations, 128-bit salt entropy, citing `django/contrib/auth/hashers.py`, from 8 files out of 400.

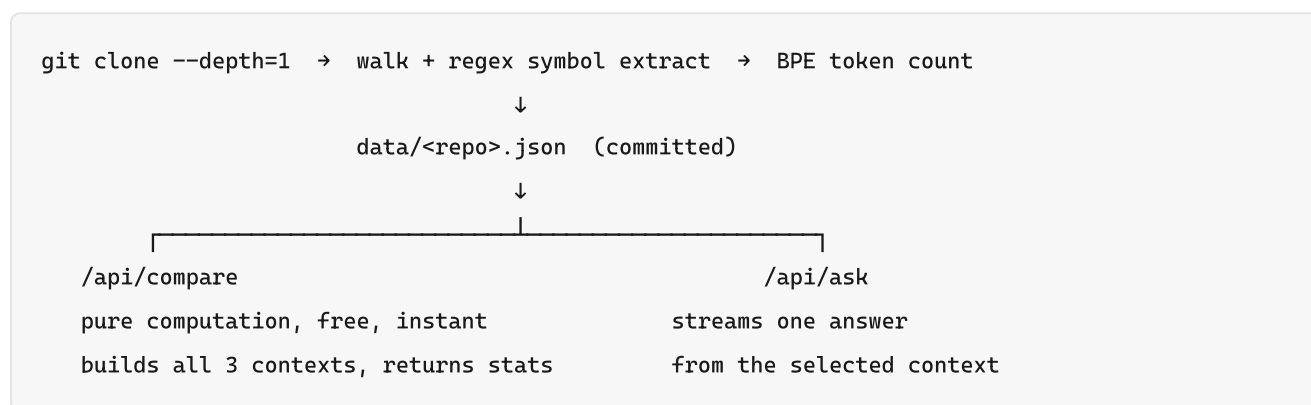
The thing I did not expect

I went in thinking the story would be "folded uses fewer tokens." It doesn't. Folded and retrieval land on almost exactly the same number, because they are both given the same 40k budget. That is not a bug, it is the actual finding.

The difference is what they spend it on. Retrieval spends everything on depth and goes blind to 383 files — the model never learns they exist. Folded spends part of the budget on a repo-wide skeleton so nothing ever fully disappears, then spends the rest on depth. It gets fewer files at full source (10 vs 17) and still answers correctly.

So the honest framing is not "compression is better." It is: **at a fixed budget, you are choosing between depth and awareness, and most tools silently choose depth without telling you.** That reframing only happened because I built the measurement first and looked at the numbers, instead of deciding the conclusion up front.

2. Architecture



Indexing is build-time, never at request time. This is the decision I care most about. A serverless function cannot clone and parse Django inside a request timeout. More importantly, a demo that indexes live is a demo that can fail while a founder is clicking it. So repos are cloned shallow, parsed, counted once, and the resulting JSON is committed to the repo. A cold Vercel deploy serves it instantly and there is nothing to go wrong.

The index is one JSON per repo: for each file, its path, language, byte size, token count, extracted symbols (name, kind, line, signature) and imports. Content is stored too, since the strategies need it to assemble context.

Folding is three tiers, not top-k. This is the actual algorithm:

1. Every file contributes its symbol names to a repo-wide skeleton. This is charged against the budget **first**, before anything else. That guarantees nothing is ever fully invisible.
2. Whatever budget remains is spent depth-first on the highest-scoring files: full source for the strongest matches, signatures only for their neighbours.
3. Everything else stays at name-only resolution.

Relevance is import-graph aware. A file imported by a high scorer inherits 35% of that score. This is the one thing keyword or vector retrieval structurally cannot do — it pulls in files that never mention your query terms but are on the path. In the Django run, 8 files entered the full-source tier purely through the graph.

Two models, deliberately separate. Token counts use the `o200k_base` BPE tokenizer as a stand-in for Claude's. Dollar figures are projected at `claude-opus-5` rates, because that is the benchmark I am reproducing. Answers are generated by `gemini-3.7-flash` on the free tier so the demo can be clicked without a bill. The UI labels which number is which — I did not want a demo that quietly implies one model did all three jobs.

3. Decisions, including the ones I got wrong

This is the section I'd most want to be asked about, so I'm including the reversals.

GitHub API → `git clone --depth=1`. My first indexer pulled files through the GitHub API. Unauthenticated that is 60 requests an hour and Django needs hundreds. A shallow clone is one command, no auth, no rate limit. This was the right call from the start and I just hadn't thought about it.

char/3.5 → **real BPE tokenizer**. I estimated tokens as characters divided by 3.5. It gave 540,595 for Django. The real BPE count is 408,333 — my estimate was **32% high**. I only caught this because I swapped in a real tokenizer for an unrelated reason. For an app whose entire claim is token accounting, shipping a number that wrong would have been embarrassing. Lesson I'd repeat: if a number is your headline, measure it, don't approximate it.

Anthropic → **Gemini for answers**. I didn't have an Anthropic API key. I picked Gemini over Groq specifically because Gemini's 1M context can hold the 408k naive case — on Groq the naive comparison would just fail, and that comparison is the whole point. The cost of this decision is that I now price in Claude tokens and answer with Gemini, which I handle by labelling both clearly rather than hiding it.

Naive answers are gated behind a confirm. One click sends the entire repo — \$2.04 at Claude rates, and enough tokens to exhaust the Gemini free tier in a single call. It would be easy to let people click it and let the demo break. The confirm dialog says exactly what it will cost before it runs.

Keyword retrieval, not embeddings — on purpose. I could have added a vector store. I decided not to, because embeddings change *recall*, not the structural property I'm demonstrating. A vector search still returns top-k and is still blind to everything below the cut. Adding embeddings would have made the baseline slightly stronger and the argument no different. I'd rather ship the honest baseline than a fancier one that proves the same thing.

Regex symbol extraction, not tree-sitter. For a skeleton I need names and signatures, not a correct parse tree. Regex gets that for JS/TS/Python/Go in about 30 lines. tree-sitter would be more accurate and would have cost me hours I didn't have.

A concurrency bug I caused and fixed. Three Answer buttons, one answer panel, no ownership rule. Clicking more than one interleaved several streams into the same state and produced garbage — sentences from four different answers spliced together mid-word. The streams never conflicted at the API level; they conflicted over shared UI state. Fix was to make ownership explicit: newest click aborts the previous request, and late chunks from a superseded stream are dropped rather than appended.

I'm including this because it is the same class of problem as my UI criticism below. An interface that looks simple because it hides concurrent state the user is actually responsible for.

What I deliberately did not build: auth, a database, user accounts, live arbitrary-repo indexing, embeddings, streaming polish, tests beyond one self-check. One day. The self-check (`scripts/check.ts`) asserts the things that would silently break: budget is respected, folded covers every file, relevant files actually rank into the full tier, the import graph pulls in something lexically unrelated.

4. What I'd change or add next (3A)

1. Make the token saving visible per request. This is my strongest recommendation and it comes directly from building this. Superbrain's headline is 60–80% fewer tokens. As a user I could not find anywhere that shows me that number on my own query. I am asked to trust the differentiating claim with no instrument to check it. Every request should show what naive would have cost, what it actually cost, and what got included or dropped. It is the cheapest trust-building feature available and nobody ships it.

2. Tie that to BYOK, and make it the pitch. The thing I liked most in the product is that I can bring API keys from other providers. That is strategically underused right now. If I'm using my own key, an 80% token reduction lands directly in *my* bill, and I can verify it against my provider dashboard. That is a much stronger pitch than "our plan is cheaper," because it's checkable. Right now the BYOK feature and the compression claim sit in the product as two unrelated things. They're the same story.

3. Show index freshness. See 3B — the local manifest went stale and nothing told me.

4. Fold at the symbol level, not the file level. This is my own next step and I think it applies to Superbrain too. Right now my tiers are per-file: a file is full source, or signatures, or names. But relevance isn't file-shaped. In a 2,000-line module I usually need three functions. The next version should expand individual symbols and their callees, which should cut the full-source tier by a lot again.

5. Approval UI for multi-file edits. They advertise dependency-aware multi-file refactoring plus "no file changes without approval." Those two features are in tension. A 12-file approval in a terminal collapses into rubber-stamping, which quietly defeats the safety feature they're selling.

5. UI issues I dislike (3B)

My honest first impression was that the UI is simple and easy to understand, and I liked that. Working with it for a day, I think the simplicity is hiding things I need.

1. It's simple because it isn't showing me what it knows. The only thing Superbrain persisted about my project was `.superbrain/manifest.md`, and this is the whole file content, not an excerpt:

```
Project: opengigantic
Total Files: 15
Languages: svg(5), json(2), ts(2), tsx(2), md(1), mjs(1), css(1), ico(1)

app/page.tsx    3.0 KB    [source]
next.config.ts  133 B    [config]
...
```

Path, size, type tag. No symbols, no signatures, no imports, no content. For a product selling "compresses and prioritizes code intelligence," the artifact on disk is a directory listing.

I left this file committed in the repo at `.superbrain/manifest.md` so you can check it yourself rather than take my word for it.

I want to be careful here: this is only what's stored *locally*. The real work might happen per-request in memory, or on their servers. But that's exactly my complaint — **I can't tell which, and the two possibilities are very different**. One costs me latency on every query. The other means my code leaves my machine, which is the first question any enterprise buyer asks. Nothing in the UI answers this.

2. Nothing tells me whether the index reflects my current code. The manifest describes my project at 15 files. By the end of the day it had 27 files and every source file had been rewritten. Whether or not the index updates on its own, the interface never showed me what state it was in — no timestamp, no file count, no "re-index" prompt, nothing that would let me notice the gap.

That matters more than it sounds. If the index is stale I don't get an error, I get a confident answer about code that no longer exists. On a codebase I'm actively changing — which is the only kind I'd use this on — a silent staleness is worse than a slow re-index, because I'll trust the wrong answer.

3. It's not clear whether I'm supposed to drive or supervise. It feels like Devin and VS Code at the same time. Devin is "go do it and come back," VS Code is "I'm driving." Superbrain sits between them and doesn't tell me which mode I'm in, so I don't know how much to trust a result before checking it. That uncertainty costs more time than a slower tool with clearer boundaries would.

4. I could not find the token saving anywhere in the interface. This ties the three above together. The 60–80% reduction is the product's headline claim, and during normal use I never saw a number that would let me confirm it was happening on my query. If it is surfaced somewhere, it isn't surfaced where the work happens. That is the gap the tool I built is aimed at, and it's why I built a measurement instrument rather than another agent.

6. Honest limitations

Things I'd want to say before someone finds them:

- **400-file cap.** Django has 783 code files; I index 400. The comparison is real but it is not the whole repo.

- **Folded is not universally better.** Retrieval gets *more* files at full source (17 vs 10. in the same budget. For a narrow question where the answer sits entirely inside two files, retrieval's extra depth could beat folding. I have not found the crossover point and I would want to before claiming folding always wins.
 - **Scoring is lexical.** Import-graph propagation helps, but the base score is still keyword matching.
 - **Regex symbol extraction misses things** — decorators, nested classes, dynamic definitions.
 - **The tokenizer is a proxy.** `o200k_base` is not Claude's tokenizer. Absolute counts are off by a few percent. The naive:folded *ratio* is essentially tokenizer-independent, which is why I'm comfortable reporting it.
 - **Two repos tested.** Django and Express.
-
-

7. Links

- **Repo:** <https://github.com/mdsvr/tokenfold-bench>
- **Live:** <https://tokenfold-bench.vercel.app>
- **Try it:** pick `django/django`, hit Compare, then look at the "invisible" row on each card and the tile map below. Then hit Answer on the folded card.